

Frontend Developer Guide: n2n-image-processing

Project Name: n2n-image-processing (Frontend)

Framework: React + Vite + React Flow

Introduction & Overview

1.1 โปรเจกต์คืออะไร? (What is this?)

n2n-image-processing คือ Web Application ที่ช่วยให้ผู้ใช้งานสามารถสร้างและทดสอบกระบวนการประมวลผลภาพ (Image Processing Pipelines) ได้ผ่านหน้าจอแบบกราฟิก (GUI) โดยใช้หลักการ

Node-based Interface

ผู้ใช้งานสามารถ:

- ลากและวางกล่องเครื่องมือ (Nodes) เช่น การโหลดภาพ, การปรับแต่งภาพ, หรือการตรวจจับวัตถุ
- เชื่อมต่อเส้น (Edges) เพื่อกำหนดลำดับการทำงาน
- กดปุ่ม "Run" เพื่อดูผลลัพธ์การประมวลผลได้ทันทีแบบ Real-time โดยระบบจะส่งคำสั่งไปทำงานที่ Python Backend

1.2 สถาปัตยกรรมระบบ (System Architecture)

ส่วน Frontend ทำหน้าที่เป็น "Visual Orchestrator" หรือตัวควบคุมการทำงาน โดยมีการติดต่อสื่อสารกับระบบดังนี้:

- User Interface: สร้างด้วย React และ React Flow เพื่อจัดการกราฟ
- API Communication: ใช้ Axios ส่งข้อมูล JSON (Parameters) และ File ไปยัง Backend (FastAPI)
- Result Rendering: รับ URL ของรูปภาพผลลัพธ์จาก Backend มาแสดงผลบน Node

2. เทคโนโลยีที่ใช้ (Tech Stack)

คนที่จะมารับงานต่อ จำเป็นต้องมีความรู้พื้นฐานในเครื่องมือต่อไปนี้:

Core Frameworks

- [React 19](#): ไลบรารีหลักสำหรับสร้าง UI versionล่าสุด (^19.1.0) เขียนด้วย TypeScript (~5.8.3)
- [Vite 7](#): เครื่องมือ Build tool ยุคใหม่ที่มีประสิทธิภาพสูงมาก (^7.0.4) ใช้สำหรับรัน Local server และ Build production

Workflow & Graphics

- [React Flow \(v11\)](#): หัวใจสำคัญของโปรเจกต์ ใช้สร้างระบบ Node-based Editor (^11.11.4)
- Lucide React: ชุดไอคอนที่ใช้ในโปรเจกต์ (^0.563.0)

Styling

- [Tailwind CSS](#): ใช้จัดหน้าตาเว็บไซต์แบบ Utility-first version 3 (^3.4.19)

Connectivity & Tools

- Axios: ใช้สำหรับยิง API Request ไปยัง Backend (^1.13.2)

- Sucrase: ใช้เพื่อการ Compile ที่รวดเร็ว (^3.35.1)
- ESLint 9: เครื่องมือตรวจสอบคุณภาพโค้ดมาตรฐานใหม่ (^9.30.1)

3. โครงสร้างโปรเจก (Project Structure)

โครงสร้างไฟล์ของ src/ แบ่งออกตามหน้าที่การทำงาน (Functionality) โดยละเอียด ดังนี้:

3.1 โฟลเดอร์: Components (ส่วนแสดงผลและ UI)

จัดการทุกสิ่งที่ผู้ใช้งานมองเห็นและโต้ตอบด้วย

- src/components/nodes/ (Custom Nodes)
 - หน้าที่: เก็บไฟล์ "กล่อง Node" ทุกประเภทที่ใช้ในระบบ
 - ไฟล์ภายใน: SiftNode.tsx, BrisqueNode.tsx, ImageInputNode.tsx, SaveImageNode.tsx, และอื่นๆ
- src/components/modals/ & common/
 - หน้าที่: เก็บหน้าต่าง Popup และ UI ที่ใช้ร่วมกัน
 - ไฟล์สำคัญ:
 - AlgorithmInfoModal.tsx: หน้าต่างแสดงคำอธิบายของแต่ละอัลกอริทึม
 - Modal.tsx: Component หน้าต่าง Popup พื้นฐาน
- src/components/sidebar.tsx
 - หน้าที่: เมนูด้านซ้ายมือสำหรับเลือกและลาก Node ลงมาวาง (Drag & Drop)
- src/components/FlowCanvas.tsx
 - หน้าที่: พื้นที่กระดานวาดกราฟ (Wrapper ของ React Flow) ทำหน้าที่รองรับการวาง Node และจัดการเส้นเชื่อม
- src/components/WorkflowControls.tsx
 - หน้าที่: แถบปุ่มควบคุมด้านบนขวา (ปุ่ม Run, Clear, Save, Load)
- src/components/WorkflowTabs.tsx
 - หน้าที่: แถบจัดการ Tab ด้านบนซ้าย สำหรับสลับหรือสร้าง Workflow ใหม่
- src/components/LogPanel.tsx
 - หน้าที่: แถบแสดงสถานะการทำงาน (Log) ด้านล่างจอ เพื่อบอกผู้ใช้งานว่ากำลังประมวลผลอะไรอยู่

3.2 โฟลเดอร์: Lib (ตรรกะหลักและการเชื่อมต่อ Backend)

ส่วนสมองของระบบที่ทำงานเบื้องหลัง

- src/lib/runners/ (API Connectors)
 - หน้าที่: รวบรวมข้อมูลจาก Node และส่งไปยัง Backend (FastAPI)
 - ไฟล์ภายใน: แยกตามหมวดหมู่ เช่น features.tsx, quality.tsx, restoration.tsx, segmentation.tsx
 - ไฟล์เสริม: utils.tsx (ฟังก์ชันช่วยแปลงข้อมูล) และ saver.ts (Logic การบันทึกข้อมูล)
- src/lib/templates/ (Presets Data)

- หน้าที: เก็บข้อมูล JSON ของกราฟตัวอย่าง (Templates)
 - ไฟล์ภายใน: alignment.ts, classification.ts, enhancement.ts ฯลฯ
- src/lib/flowConfig.ts (Node Registry)
 - หน้าที: สำคัญมาก เป็นไฟล์ลงทะเบียนจับคู่ "ชื่อ Node" กับ "ไฟล์ Component" (ถ้าย้ายหรือสร้าง Node ใหม่ ต้องแก้ที่นี่)
- src/lib/workflowTemplates.ts (Template Registry)
 - หน้าที: ไฟล์ลงทะเบียนรายชื่อ Template ทั้งหมดที่จะไปโชว์ในเมนูคลิกขวา
- src/lib/validation.ts
 - หน้าที: กฎการตรวจสอบกราฟก่อนรัน (เช่น เช็คว่เชื่อมสายครบไหม)
- src/lib/api.ts
 - หน้าที: การตั้งค่า Axios และ Base URL ของ Server

3.3 โฟลเดอร์: Hooks & Context (State Management)

ฟังก์ชันพิเศษสำหรับจัดการข้อมูลในแอป

- src/RunContext.tsx (Core Engine)
 - หน้าที: ตัวควบคุมลำดับการทำงาน (Orchestrator) ทำหน้าที่สั่งรัน Node ทีละตัวตามลำดับ
- src/hooks/useFlowHistory.ts
 - หน้าที: ระบบ Undo/Redo (ย้อนกลับการแก้ไข)
- src/hooks/useNodeStatus.ts
 - หน้าที: จัดการสถานะ Loading (หมุนๆ) และ Error ของ Node
- src/hooks/useFlowHotkeys.ts
 - หน้าที: จัดการปุ่มลัดคีย์บอร์ด (เช่น กด Delete เพื่อลบ Node, Ctrl+Z เพื่อ Undo)
- src/hooks/useWorkflowFile.ts
 - หน้าที: Logic การบันทึกกราฟเป็นไฟล์ .json และโหลดกลับเข้ามา

3.4 ไฟล์ระดับ Root (Configuration)

ไฟล์ตั้งค่าพื้นฐานที่ root ของโปรเจค

- src/types.ts
 - หน้าที: เก็บ TypeScript Interface กลางของทั้งโปรเจค (เช่น โครงสร้างข้อมูลของ Node, Edge, API Response)
- src/App.tsx & main.tsx
 - หน้าที: จุดเริ่มต้นของ Application และการวาง Layout หลัก

4. การสร้าง Node ใหม่ (Creating a Custom Node)

การเพิ่มอัลกอริทึมใหม่เข้าสู่ระบบ จะต้องทำการแก้ไขไฟล์ใน 5 จุดหลัก เพื่อให้ Node ปรากฏบนหน้าจอ, ลากวางได้, และทำงานได้จริง

Step 1: สร้างไฟล์ Component ของ Node

สร้างไฟล์ .tsx ใหม่ในโฟลเดอร์ src/components/nodes/ (แนะนำให้ Copy จากไฟล์ที่มีอยู่แล้ว เช่น SiftNode.tsx มาเป็นต้นแบบ)

Step 2: ลงทะเบียน Node (Register Node)

บอกให้ React Flow รู้จักชื่อและไฟล์ของ Node ใหม่

- File Location: src/lib/flowConfig.ts
- Action: เพิ่ม Key ใหม่เข้าไปใน object nodeTypes

```
export const nodeTypes: NodeTypes = {  
  'image-input': ImageInputNode,  
  sift: SiftNode,  
  surf: SurfNode,  
  orb: OrbNode,  
  brisque: BrisqueNode,  
  psnr: PsnrNode,  
  ssim: SsimNode,  
  bfmatcher: BFMatcherNode,  
  flannmatcher: FLANNMatcherNode,  
  'homography-align': HomographyAlignNode,  
  'affine-align': AffineAlignNode,  
  otsu: OtsuNode,  
  snake: SnakeNode,  
  'save-image': SaveImageNode,  
  'save-json': SaveJsonNode,  
  
  'clahe': CLAHENode,  
  
  'msrcr': MSRCRNode,  
  
  'zero': ZeroDCENode,  
  'zeroDce': ZeroDCENode,  
  
  'dcnn': DnCNNNode,  
  'dncnn': DnCNNNode,  
  
  'real': RealESRGANNode,  
  'realesrgan': RealESRGANNode,  
  
  'swinir': SwinIRNode,  
  
  'deep': DeepLabNode,  
  'deeplab': DeepLabNode,  
  
  'mask': MaskRCNNNode,  
  'maskrcnn': MaskRCNNNode,  
  
  'unet': UNetNode,  
};
```

Step 3: เพิ่มปุ่มใน Sidebar (Add to Menu)

สร้างปุ่มให้ผู้ใช้สามารถลาก Node นี้ลงไปใน Canvas ได้

- File Location: src/components/sidebar.tsx
- Action: เพิ่ม div ที่มี event onDragStart

```
const onDragStart = (e: React.DragEvent, nodeType: string) => {
  e.dataTransfer.setData('application/reactflow', nodeType);
  e.dataTransfer.effectAllowed = 'move';
};

const jobs = useMemo(() => ([
  {
    name: 'Input',
    headerColor: 'text-teal-400',
    algorithms: [{ type: 'image-input', label: 'Image Input', color: 'bg-teal-600' }]
  },
  {
    name: 'Enhancement',
    headerColor: 'text-indigo-500',
    algorithms: [
      { type: 'clahe', label: 'CLAHE', color: 'bg-indigo-500' },
      { type: 'msrcr', label: 'MSRCR Retinex', color: 'bg-indigo-500' },
      { type: 'zeroDce', label: 'Zero-DCE Lighten', color: 'bg-indigo-500' }
    ]
  },
  {
    name: 'Restoration',
    headerColor: 'text-red-500',
    algorithms: [
      { type: 'dncnn', label: 'DnCNN Denoise', color: 'bg-red-500' },
      { type: 'realesrgan', label: 'Real-ESRGAN', color: 'bg-red-500' },
      { type: 'swinir', label: 'SwinIR Transformer', color: 'bg-red-500' }
    ]
  },
  {
    name: 'Segmentation',
    headerColor: 'text-yellow-500',
    algorithms: [
      { type: 'deeplab', label: 'DeepLab v3+', color: 'bg-yellow-500' },
      { type: 'maskrcnn', label: 'Mask R-CNN', color: 'bg-yellow-500' },
      { type: 'unet', label: 'U-Net', color: 'bg-yellow-500' }
    ]
  }
]),
```

Step 4: เชื่อมต่อ API (Create Runner Logic)

เมื่อ Node ถูกสร้างขึ้นมาแล้ว มันยังเป็นแค่กล่องเปล่าๆ เราต้องเขียนฟังก์ชัน "Runner" เพื่อบอกว่าเมื่อถึงคิว Node นี้มัน จะทำอะไร (ส่งข้อมูลอะไรไปหา Backend)

- ตำแหน่งไฟล์: src/lib/runners/
 - แนะนำให้สร้างไฟล์ใหม่สำหรับกลุ่มอัลกอริทึมของคุณ หรือเพิ่มต่อท้ายไฟล์เดิมที่มีอยู่แล้ว (เช่น features.tsx)

สิ่งที่ต้องทำในไฟล์ Runner:

1. Validate: ตรวจสอบว่า Node ก่อนหน้าส่งรูปมาให้หรือยัง?
2. Status Update: เปลี่ยนสถานะ Node เป็น "Running" (เพื่อให้ขึ้นไฟหมุนๆ)
3. API Call: ยิง Request ไปยัง Backend
4. Result Update: รับ URL รูปผลลัพธ์จาก Backend แล้วอัปเดตกลับไป Node

```
export async function runFeature(  
  node: RFNode,  
  setNodes: SetNodes,  
  nodes: RFNode[],  
  edges: Edge[]  
) {  
  const nodeId = node.id;  
  
  // --- Helper: ฟังก์ชันสำหรับแจ้ง Error และเปลี่ยนสถานะเป็นสีแดง ---  
  const fail = async (msg: string) => {  
    await updateNodeStatus(nodeId, 'fault', setNodes);  
    throw new Error(msg);  
  };  
  
  // -----  
  // 1. Validation Logic: ตรวจสอบความถูกต้องของ Input  
  // -----  
  const incoming = edges.filter((e) => e.target === nodeId);  
  if (incoming.length === 0) {  
    return fail('No input connection (Please connect an Image source).');  
  }  
  
  // ป้องกันการต่อสายผิดประเภท (เช่น เอา Output ของ SIFT ตัวอื่นมาเข้า SIFT ตัวนี้)  
  const prevNode = nodes.find((n) => n.id === incoming[0].source);  
  const BAD_SOURCES = ['sift', 'surf', 'orb', 'bfmatcher', 'otsu', 'save-json']; // รายชื่อ Node ที่ห้ามเป็น Input  
  
  if (prevNode && BAD_SOURCES.includes(prevNode.type || '')) {  
    return fail('Invalid Input: Feature Extraction requires a raw Image source.');  }  
  
  // ค้นหาภาพต้นฉบับจาก Node แม่ข่าย (Traverse back to find image)  
  const imagePath = findInputImage(nodeId, nodes, edges);  
  if (!imagePath) {  
    return fail('No input image found.');  }  
}
```

```

// -----
// 2. Dispatcher: เลือกฟังก์ชัน API ตามประเภท Node
// -----
let prefix = '';
let runner: any;

switch (node.type) {
  case 'sift': prefix = 'SIFT'; runner = runSift; break;
  case 'surf': prefix = 'SURF'; runner = runSurf; break;
  case 'orb': prefix = 'ORB'; runner = runOrb; break;
  default: return;
}

// อัปเดต UI ให้ user เห็นว่ากำลังทำงาน
await markStartThenRunning(nodeId, `Running ${prefix}`, setNodes);

try {
  // -----
  // 3. Execution: ยิง API ไปยัง Backend
  // -----
  const params = node.data.payload?.params;
  const resp = await runner(imagePath, params); // <--- จุดที่ยิง API จริง

  // -----
  // 4. Result Handling: จัดการผลลัพธ์ที่ได้
  // -----
  // Normalize ข้อมูลบางตัวเพื่อให้ใช้ง่าย
  const num_keypoints = resp.num_keypoints ?? resp.kps_count ?? 0;
  const visUrl = resp.vis_url ? abs(resp.vis_url) : undefined; // แปลง Path เป็น Absolute URL

  // อัปเดตข้อมูลกลับลงไปใน Node (React State)
  setNodes((nds) =>
    nds.map((n) =>
      n.id === nodeId
        ? {
            ...n,
            data: {
              ...n.data,
              status: 'success', // เปลี่ยนไฟเป็นสีเขียว
              description: `Found ${num_keypoints} keypoints`, // ข้อความโชว์ได้กล่อง

              // สำคัญ: เก็บ Response ทั้งหมดลง Payload เพื่อให้ Node ถัดไปดึงไปใช้ได้
              payload: {
                ...(n.data as CustomNodeData)?.payload,
                ...resp,
                vis_url: visUrl,
                output_image: visUrl, // Standard field สำหรับรูปผลลัพธ์
                json_url: resp.json_url
              },
            } as CustomNodeData,
          }
        : n
    )
  );
} catch (err: any) {
  console.error(`${prefix} Error:`, err);
  await fail(err?.message || `${prefix} failed`);
}

```

รูปแบบการส่งข้อมูล (Data Payload Protocol) ในโปรเจกต์นี้ Node ส่วนใหญ่ (ที่เป็น Processing Node) จะใช้วิธีส่งข้อมูลแบบ JSON ไปยัง Backend โดยมีโครงสร้างมาตรฐานดังนี้:

1. image_path (String): ที่อยู่ของไฟล์รูปภาพ (ซึ่งมักจะได้อาจมาจาก Output ของ Node ก่อนหน้า) ไม่ได้ส่งไฟล์ Binary ไปตรงๆ
2. params (Object): ค่าพารามิเตอร์ต่างๆ ที่ User ตั้งค่าใน UI (เช่น nfeatures, threshold)

ตัวอย่างโค้ด (จากไฟล์ src/lib/api.ts):

```
export async function runSift(image_path: string, params?: Record<string, any>, signal?: AbortSignal) {
  const resp = await fetch(`${API_BASE}/api/feature/sift`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ image_path, params }),
    signal,
  });
  return handleResponse(resp);
}
```

Step 5: เพิ่มเข้าสู่ Loop การทำงาน (Update Execution Loop)

ขั้นตอนนี้สำคัญมาก คือการบอก "ระบบควบคุมกลาง" (FlowCanvas.tsx) ให้รู้จักและเรียกใช้ Runner ที่เราเพิ่งเขียนไปใน Step 4

- ตำแหน่งไฟล์: src/FlowCanvas.tsx

สิ่งที่ต้องทำ:

1. Import Runner เข้ามาก่อน ที่ส่วนบนสุดของไฟล์ src/FlowCanvas.tsx:

```
// ----- Runners -----
import { runFeature } from './lib/runners/features';
import { runQuality } from './lib/runners/quality';
import { runMatcher } from './lib/runners/matching';
import { runAlignment } from './lib/runners/alignment';
import { runOtsu, runSnakeRunner } from './lib/runners/classification';
import { runSaveImage, runSaveJson } from './lib/runners/saver';
import { markStartThenRunning } from './lib/runners/utils';
import { runEnhancement } from './lib/runners/enhancement';
import { runRestoration } from './lib/runners/restoration';
import { runSegmentation } from './lib/runners/segmentation';
```

2. เพิ่ม Case ในฟังก์ชัน runNodeById ค้นหาฟังก์ชันที่ชื่อ runNodeById (ประมาณบรรทัด 130+) แล้วเพิ่ม case ใหม่เข้าไปใน switch:

```
switch (typeKey) {
  case 'image-input':
    if (!node.data.payload?.url) throw new Error("No image uploaded yet.");
    setNodes(nds => nds.map(n => n.id === nodeId ? { ...n, data: { ...n.data, status: 'success' as NodeStatus } } : n));
    await new Promise(r => setTimeout(r, 100));
    break;

  case 'sift': case 'surf': case 'orb': await runFeature(node, setNodes, freshNodes, freshEdges); break;
  case 'brisque': case 'psnr': case 'ssim': await runQuality(node, setNodes, freshNodes, freshEdges); break;
  case 'bfmatcher': case 'flannmatcher': await runMatcher(node, setNodes, freshNodes, freshEdges); break;
  case 'homography-align': case 'affine-align': await runAlignment(node as any, setNodes as any, freshNodes, freshEdges); break;
  case 'otsu': await runOtsu(node as any, setNodes as any, freshNodes, freshEdges); break;
}
```

3. (Optional) จัดลำดับความสำคัญ (Execution Priority) หาก Node ของคุณจำเป็นต้องรันก่อนหรือหลังคนอื่นเป็นพิเศษ ให้ไปเพิ่มชื่อในตัวแปร executionPriority ภายใน useEffect

```
const executionPriority: Record<string, number> = {
  'image-input': 1,
  'dcnn': 10, 'dncnn': 10, 'swinir': 10, 'real': 10, 'realesrgan': 10,
  'clahe': 20, 'msrcr': 20, 'zero': 20, 'zerodce': 20, 'zero_dce': 20,
  'otsu': 30, 'snake': 30,
  'deep': 35, 'deeplab': 35, 'unet': 35, 'mask': 35, 'maskrcnn': 35,
  'sift': 40, 'surf': 40, 'orb': 40,
  'bfmatcher': 50, 'flannmatcher': 50,
  'homography-align': 60, 'affine-align': 60,
  'brisque': 90, 'psnr': 90, 'ssim': 90,
  'save-image': 99,
  'save-json': 99
};
```

5. ระบบแม่แบบ (Workflow Templates)

ระบบ Template ช่วยให้ผู้ใช้สามารถโหลด Workflow สำเร็จรูปได้ทันที โดยระบบนี้ประกอบด้วย 3 ส่วนหลักที่ทำงานร่วมกัน คือ การนิยามไฟล์ (Definition), การลงทะเบียน (Registration), และ การแสดงผล (UI Integration)

5.1 โครงสร้างไฟล์ Template (Template Definition)

File Location: src/lib/templates/*.ts (แยกไฟล์ตามหมวดหมู่)

Template แต่ละอันคือ Object ภาษา TypeScript ที่เก็บข้อมูลโครงสร้างของกราฟทั้งหมด (Nodes & Edges)

5.2 การลงทะเบียน Template (Template Registration)

File Location: src/lib/workflowTemplates.ts

เมื่อสร้างไฟล์ Template เสร็จแล้ว ต้องนำมา "ลงทะเบียน" เพื่อให้ระบบรู้จัก
ขั้นตอนการทำ:

1. เปิดไฟล์ src/lib/workflowTemplates.ts
2. Import ตัวแปร Template ที่เพิ่งสร้างเข้ามา
3. เพิ่มเข้าไปใน Array TEMPLATES

```
// --- Import Templates ---

// 1. Feature & Matching
import { FEATURE_EXTRACTION_SINGLE } from './templates/feature';
import { FEATURE_MATCHING_PIPELINE } from './templates/matching';

// 2. Classification
import {
  OTSU_CLASSIFICATION_TEMPLATE,
  SNAKE_CLASSIFICATION_TEMPLATE,
} from './templates/classification';

// 3. Alignment
import { OBJECT_ALIGNMENT_HOMOGRAPHY } from './templates/alignment';

// 4. Quality
import {
  PSNR_ASSESSMENT_TEMPLATE,
  BRISQUE_ASSESSMENT_TEMPLATE,
} from './templates/quality';

// 5. NEW TEMPLATES
import { ENHANCEMENT_CLAHE_TEMPLATE } from './templates/enhancement';
// 6. Restoration
import { RESTORATION_REALESRGAN_TEMPLATE } from './templates/restoration';
// 7. Segmentation
import { SEGMENTATION_DEEPLAB_TEMPLATE } from './templates/segmentation';
```

```
export const TEMPLATES: WorkflowTemplate[] = [
  // Existing
  FEATURE_EXTRACTION_SINGLE,
  FEATURE_MATCHING_PIPELINE,
  PSNR_ASSESSMENT_TEMPLATE,
  BRISQUE_ASSESSMENT_TEMPLATE,
  OTSU_CLASSIFICATION_TEMPLATE,
  SNAKE_CLASSIFICATION_TEMPLATE,
  OBJECT_ALIGNMENT_HOMOGRAPHY,
  ENHANCEMENT_CLAHE_TEMPLATE,
  RESTORATION_REALESRGAN_TEMPLATE,
  SEGMENTATION_DEEPLAB_TEMPLATE,
];
```

5.3 การแสดงผลและการจัดหมวดหมู่ (Sidebar & Categorization)

File Location: src/components/sidebar.tsx

ระบบ Sidebar มีความฉลาดในการ "จัดหมวดหมู่อัตโนมัติ" โดยดูจากค่า color ที่ระบุไว้ใน Template ไฟล์

ตารางการจับคู่สีกับหมวดหมู่ (Color Mapping): อ้างอิงจาก Logic ในไฟล์ sidebar.tsx

สี (Color Property)	หมวดหมู่ใน Sidebar (Category)
'indigo'	Enhancement
'red'	Restoration
'yellow'	Segmentation
'green'	Feature Extraction
'orange'	Matching
'purple'	Object Alignment
'pink'	Classification
'blue'	Quality Assessment
(อื่นๆ)	Other

- กรณีปกติ: หาก Template ใหม่ของคุณอยู่ในหมวดหมู่เดิมที่มีอยู่แล้ว ให้เลือกใช้สีตามตารางข้างบน แล้วระบบจะนำไปใส่ใน Folder นั้นให้เองทันที
- กรณีพิเศษ (สร้างหมวดหมู่ใหม่): หากต้องการเพิ่มหมวดหมู่ใหม่ (เช่น "3D Reconstruction") ให้ทำดังนี้:
 - ตั้งสีใหม่ใน Template (เช่น color: 'cyan')
 - แก้ไฟล์ src/components/sidebar.tsx ในฟังก์ชัน templateJobs ให้เพิ่มเงื่อนไข else if (t.color === 'cyan') { ... }

6. ระบบตรวจสอบความถูกต้อง (Validation System)

ระบบ Validation ถูกเขียนไว้ที่ไฟล์ src/lib/validation.ts โดยแบ่งการตรวจสอบออกเป็น 2 ระดับ คือ Structure Validation (โครงสร้างกราฟ) และ Logic Validation (ค่าพารามิเตอร์)

6.1 โครงสร้างข้อมูลกฎเกณฑ์ (Validation Rules Structure)

ระบบใช้ Object ชื่อ NODE_RULES เป็นฐานข้อมูลกลางในการตรวจสอบ Input/Output

- Inputs (string[]): รายการประเภทข้อมูลที่ Node นั้น ต้องการ (เช่น ['image'] หรือ ['feature', 'feature'])
- Output (string): ประเภทข้อมูลที่ Node นั้น ส่งออกไป (เช่น 'match' หรือ 'metric')

Technical Note:

- any: เป็น Wildcard พิเศษที่รับ Input อะไรก็ได้ (ใช้กับ Node พวก Saver เช่น save-json)
- none: ใช้ระบุว่า Node นี้ไม่มี Output ส่งต่อ (เช่น save-image ถือเป็นจุดสิ้นสุด)

6.2 การป้องกัน Loop (Cycle Detection)

ฟังก์ชัน hasCycle ใช้ป้องกันไม่ให้ผู้ใช้ลากเส้นเชื่อมแบบวงกลม (เช่น A -> B -> A) ซึ่งจะทำให้โปรแกรมค้าง (Infinite Loop) ตอนรัน

- Algorithm: ใช้การค้นหาแบบ Depth-First Search (DFS)
- Logic:
 1. เริ่มจาก Node เป้าหมาย (Target)
 2. ไล่ย้อนกลับไปหา Node ลูกข่าย (Outgoers) ทั้งหมดแบบ Recursive
 3. ถ้าไปเจอ Node ต้นทาง (Source) อีกครั้ง แสดงว่าเกิด Cycle -> Block การเชื่อมต่อทันที

6.3 การตรวจสอบความเข้ากันได้ (Type Compatibility)

ฟังก์ชัน isTypeCompatible จะถูกเรียกตอนที่เมาส์กำลังลากเส้นไปหา Node ปลายทาง

- Logic:
 - เช็ค NODE_RULES ของ Node ต้นทาง (Source) และปลายทาง (Target)
 - ถ้า Output ของ Source ตรงกับหนึ่งใน Inputs ของ Target -> อนุญาต
 - ถ้า Inputs ของ Target มีคำว่า 'any' -> อนุญาตเสมอ

6.4 การตรวจสอบก่อนการรัน (Pre-Flight Checks)

ฟังก์ชัน validateNodeInput คือด่านตรวจที่ละเอียดที่สุด จะถูกเรียกเมื่อกดปุ่ม Run Workflow โดยมีความสามารถพิเศษในการตรวจสอบ "เนื้อใน" ของข้อมูล

A. การตรวจสอบจำนวน Input (Connectivity Check)

- ระบบจะนับจำนวนเส้นที่ต่อเข้า Node (inputCount)
- General Rule: Node ส่วนใหญ่ต้องมี Input อย่างน้อย 1 เส้น
- Specific Rule:
 - psnr, ssim: ต้องมี 2 เส้น (Original Image & Processed Image)
 - bfmatcher, flannmatcher: ต้องมี 2 เส้น (Feature A & Feature B)

B. การตรวจสอบมิติข้อมูล (Dimension Check)

สำหรับ Node ประเภท Matcher ระบบจะเจาะลึกไปถึง Parameters ของ Node ก่อนหน้า เพื่อดูว่า Feature Vector เข้ากันได้ไหม

- ฟังก์ชันช่วย: getDescriptorDimension(node)
 - SIFT: คืนค่า 128
 - ORB: คืนค่า 32
 - SURF:
 - ถ้า extended=true -> คืนค่า 128
 - ถ้า extended=false -> คืนค่า 64
- Validation Logic: ระบบจะเปรียบเทียบ Dimension ของ Input ทั้งสองตัว
 - ถ้าเอา ORB (32) มาจับคู่กับ SIFT (128) -> Error: Dimension Mismatch
 - ถ้าเอา SURF (64) มาจับคู่กับ SURF Extended (128) -> Error: Dimension Mismatch

C. การตรวจสอบรูปภาพ (Image Validity)

- สำหรับ Node image-input ระบบจะเช็คว่าคุณได้อัปโหลดรูปหรือยัง (node.data.payload.url) ถ้ายังไม่มี -> Error

7. การจัดการสถานะและ Hooks (State & Hooks)

นอกจากการใช้ State พื้นฐานของ React Flow แล้ว โปรเจกต์นี้ยังมี Custom Hooks ที่ออกแบบมาเฉพาะเพื่อจัดการสถานะที่ซับซ้อน

7.1 ระบบ Undo/Redo (useFlowHistory)

File Location: src/hooks/useFlowHistory.ts

Hook นี้ทำหน้าที่บันทึกประวัติการแก้ไขกราฟ เพื่อให้ผู้ใช้สามารถย้อนกลับ (Undo) หรือทำซ้ำ (Redo) ได้

หลักการทำงาน (Key Mechanisms):

1. Smart filtering (getCleanData): ระบบจะ ไม่บันทึก ข้อมูลขนาดใหญ่ที่ไม่จำเป็นลงใน History Stack เช่น:
 - o payload (ข้อมูลผลลัพธ์รูปภาพ)
 - o status (สถานะ Loading/Success)
 - o json_data
 - o เหตุผล: เพื่อประหยัด Memory และป้องกันการโหลดซ้ำที่ซ้ำ
2. Change Detection (hasMeaningfulChange): ระบบจะบันทึก Snapshot ใหม่ก็ต่อเมื่อมีการเปลี่ยนแปลงที่มีนัยสำคัญเท่านั้น เช่น:
 - o จำนวน Node/Edge เปลี่ยนไป
 - o ตำแหน่ง (x, y) เปลี่ยน
 - o ค่า Parameter ใน data เปลี่ยน
3. Applying History: เมื่อกด Undo ระบบจะนำ Snapshot เก่ามาทับ แต่จะ คงค่า payload และ status ปัจจุบันไว้ (สังเกตได้จากฟังก์ชัน applySnapshot ที่มีการ merge curD.payload) เพื่อให้ผลลัพธ์ไม่หายไปแม้จะกด Undo แค่อยับตำแหน่ง

7.2 สถานะของ Node (useNodeStatus)

File Location: src/hooks/useNodeStatus.ts

Hook เล็กๆ ที่ช่วยคำนวณสถานะของ Node เพื่อนำไปแสดงผล UI (เช่น ไฟสถานะสีเขียว/แดง)

Logic การคำนวณ:

- Running: เมื่อ status เป็น 'start' หรือ 'running'
- Fault: เมื่อ status เป็น 'fault' (แสดงไฟแดง)
- Success: เมื่อ status เป็น 'success' หรือ ตรวจสอบว่ามีข้อมูลผลลัพธ์ (hasResult) อยู่ใน Payload แล้ว

การนำไปใช้: Node Component ทุกตัวจะเรียกใช้ Hook นี้เพื่อตัดสินใจว่าจะโชว์ Spinner หมุนๆ

```
const { isRunning, isSuccess, isFault, statusDot } = useNodeStatus(data);
```

```
let borderColor = 'border-green-500';  
if (selected) borderColor = 'border-green-400 ring-2 ring-green-500';  
else if (isRunning) borderColor = 'border-yellow-500 ring-2 ring-yellow-500/50';
```

7.3 Global Execution State (RunContext)

File Location: src/RunContext.tsx

Context นี้ทำหน้าที่ส่งผ่านฟังก์ชัน runNode จาก FlowCanvas ลงไปให้ส่วนอื่นๆ ของแอปสามารถสั่งรัน Node ได้โดยไม่ต้อง Prop drilling

8. การติดตั้งและเริ่มใช้งาน (Installation & Setup)

ขั้นตอนการเตรียมสภาพแวดล้อม (Environment) และการรันโปรเจกสำหรับนักพัฒนา

8.1 สิ่งที่ต้องมี (Prerequisites)

ก่อนเริ่มติดตั้ง โปรดตรวจสอบว่าเครื่องของคุณมีซอฟต์แวร์ต่อไปนี้:

- Node.js: แนะนำเวอร์ชัน v18.0.0 ขึ้นไป (เพื่อรองรับ Vite และ React 19)
- npm: (ติดตั้งมาพร้อม Node.js) หรือจะใช้ yarn / pnpm ก็ได้
- Git: สำหรับ Clone โปรเจก

8.2 ขั้นตอนการติดตั้ง (Installation Steps)

1. Clone Project:

```
git clone <repository_url>
```

```
cd my-react-flow-app
```

2. ติดตั้ง Dependencies:

```
npm install
```

8.3 การรันโปรเจก (Running Development Server)

```
npm run dev
```

- เมื่อรันสำเร็จ Terminal จะแจ้ง URL (ปกติคือ http://localhost:5173)
- ให้เปิด Browser เข้าไปที่ URL ดังกล่าว

ข้อควรระวัง: โปรเจกนี้เป็นเพียงส่วน Frontend จำเป็นต้อง รัน Backend Server (FastAPI) ควบคู่กันไปด้วย เพื่อให้ฟังก์ชันการประมวลผลภาพทำงานได้ (ถ้าไม่รัน Backend หน้าเว็บจะเปิดติด แต่จะกด Run Process ไม่ได้)

8.4 การ Build เพื่อนำไปใช้จริง (Building for Production)

เมื่อพัฒนาเสร็จแล้วและต้องการนำไฟล์ไป Deploy บน Server:

```
npm run build
```